

ADVANCED AUTOMATION ENGINEERING PROGRAM

App Launch & Validation

Architecture · browser context & pages · validation

SPRINT 1

DAY 1 of 42

Jai Singh

Week 1 — UI Automation (Playwright) · Full-day session (3h teach + 5h hands-on)

Planner ref: Week 1 · Day 1 — App Launch & Validation

01 How Today Runs

SESSION MAP

A full day: a short teaching block to build the mental model, then most of the time hands-on in the IDE. Slides are reference anchors — the real work happens on screen.

09:30	 Concepts & setup	Why Playwright, architecture, context/pages, project setup & config
11:00	 First test, live	Launch, validate, assertions, evidence — demoed in the IDE
12:00	 Guided exercises	Short 'you try' tasks with checkpoints
13:30	 Lab (staged)	Build the smoke project in milestones with acceptance gates
16:30	 Debrief & recap	Common failures, evaluation mapping, next-day preview

Day 1 establishes the Playwright mental model and the validation toolkit: launch the app, confirm it loaded correctly, and capture evidence. Locators come Day 2; synchronization Day 3.



Why Playwright

Auto-wait, cross-browser, isolation



Architecture

Runner, browser, context, page



Context & pages

Isolation per test



Headless vs headed

How and when to run each



Assertions

Soft vs hard validation



Accessibility

WCAG basics (Axe arrives Day 4)

Aligned to Planner Wk1/Day1 “App Launch & Validation”. ToC today: project init/scaffold, npm deps, screenshots, WCAG basics. **Deferred:** selectors→D2, auto-wait→D3, traces/video→D5, CI/GitLab→Wk3+.

By the end of today you will be able to:

- ✓ Explain Playwright's architecture: test runner, browser, browser context and page.
- ✓ Describe how each test gets an isolated browser context, and why that matters.
- ✓ Initialise a Playwright + TypeScript project and run it headed and headless.
- ✓ Write a first test that launches the app and validates its title and URL.
- ✓ Use hard vs soft assertions appropriately, and capture a screenshot as evidence.
- ✓ Explain what web accessibility / WCAG is and why it belongs in test automation.



Auto-waiting built in

Waits for elements to be actionable and retries assertions automatically — far fewer flaky tests (we go deep on Day 3).



True cross-browser

One API drives Chromium, Firefox and WebKit, plus branded Chrome/Edge — same test, three engines.



Isolation by design

Each test runs in a fresh browser context — clean cookies and storage, parallel-safe out of the box.



Rich diagnostics

Screenshots, video and traces make debugging evidence-based (traces covered on Day 5).

Versions today: Playwright 1.60.x (stable, May 2026) · Node.js 24 LTS (program standard; Node 20 is EOL). Always pin versions in package.json.

You already know Java and OOP from college. Most of what we do here maps cleanly onto concepts you have — here is the translation.

WHAT YOU KNOW (JAVA)		WHAT WE USE HERE
Maven / Gradle	→	npm
pom.xml	→	package.json
mvn test	→	npx playwright test
JUnit / TestNG @Test	→	test('name', async () => { ... })
Assert.assertEquals (hard)	→	expect(...) (hard, stops test)
SoftAssert (TestNG)	→	expect.soft(...) (accumulates)
public class Page { }	→	export class Page { } (same OOP)
Java (statically typed)	→	TypeScript = JavaScript + types

Biggest new idea — async/await: Playwright calls are asynchronous, so you `await` them — like calling `.get()` on a Future, but inline.

Test Runner

@playwright/test — fixtures, parallelism, retries, reporters

**Browser**

Chromium · Firefox · WebKit launched and controlled over a fast protocol

**Browser Context**

Isolated session per test — own cookies, storage, cache

**Page & Locators**

Tabs and elements your test drives and validates

Following one test top to bottom makes the layers concrete:

- 1 Runner starts** `@playwright/test` reads the config and picks up your spec files.
- 2 Browser launches** The chosen engine (e.g. Chromium) is started, usually headless.
- 3 Context created** A fresh, isolated context is opened just for this test — no shared state.
- 4 Page opens** A tab (page) is created; your code drives it: goto, click, fill.
- 5 Assertions run** `expect(...)` auto-waits and verifies; failures capture evidence.
- 6 Context closes** Context is torn down; the next test starts clean again.

A browser context is a lightweight, isolated session inside one browser. Each test gets its own — like a fresh incognito window with no shared state.



Isolation

Separate cookies, localStorage and cache per context — tests cannot leak state into each other.



Fast & parallel-safe

Contexts are cheap to create, so the runner can run many in parallel without cross-talk.

```
// the built-in 'page' fixture already gives each test
// its own context — you rarely create one by hand
test('each test is isolated', async ({ page }) => { ... });
```



Toolchain

- Node.js 24 LTS (program standard)
- npm (or pnpm / yarn)
- VS Code + Playwright extension
- Git access to the project repo



Version discipline

- Pin Playwright 1.60.x in package.json
- Keep all @playwright/* packages in sync
- Commit package-lock.json
- Use npm ci for clean installs



IN THE IDE verify your toolchain before anything else — run the two commands below together.

```
node -v                # expect v24.x (program standard)
npx playwright --version # expect 1.60.x
```

10 Initialise the Project

HANDS-ON

From Java: npm $\approx$ Maven, and package.json $\approx$ pom.xml — dependency management you already know.



IN THE IDE use Option A in class (trainer repo). Option B is the manual route if you set up your own.

```
# Option A (preferred in class): trainer repo, pinned & consistent
git clone <trainer-repo-url> retail-automation
cd retail-automation
npm ci

# Option B: scaffold manually, then pin and install browsers
npm init playwright@latest
npm install -D @playwright/test@1.60.0
npx playwright install      # browser binaries
```

Scaffolding generates the config, an example spec, and pinned dependencies (next slide).

11 Project Structure

HANDS-ON

What the scaffold produces, and where your work goes:

```
retail-automation/  
├─ tests/  
│   └─ home.spec.ts  
├─ pages/          # POM (Sprint 3)  
├─ playwright.config.ts  
├─ package.json  
├─ package-lock.json  
└─ .gitlab-ci.yml  # CI (Wk3+)
```

tests/ Specs grouped by feature/journey

pages/ Page Objects added in Sprint 3

config One source of truth for runs

lockfile Pinned, reproducible installs

12 playwright.config.ts Essentials

HANDS-ON

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',      // where specs live
  reporter: 'html',        // rich local report
  use: {                   // one shared 'use' block
    baseURL: 'http://localhost:3000',
    screenshot: 'only-on-failure', // evidence
  },
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
  ],
});
```

One `use` block only — a second `use` key would silently overwrite the first. From Java: think of it as your `testng.xml`.

Same test, two ways to run the browser. You choose based on what you are doing.



Headed

- Browser window is visible
- Best for authoring & debugging
- Watch what the test does
- --headed flag



Headless

- No visible window — background
- Default in CI; faster, less resource
- Same engine, same results
- Default when no flag is given



IN THE IDE run once headed (--headed) to watch it, then headless to feel the speed difference.

14 First Test: Launch & Validate

HANDS-ON



IN THE IDE create tests/home.spec.ts and build this with me line by line.

```
import { test, expect } from '@playwright/test';

test('app launches and loads correctly', async ({ page }, testInfo) => {
  await page.goto('/'); // launch
  await expect(page).toHaveTitle(/Retail Store/); // title
  await expect(page).toHaveURL(/\/$/); // URL (adjust to app)
  await expect(page.getByRole('heading', { name: /Retail Store/i })).toBeVisible();
  await page.screenshot({ path: testInfo.outputPath('home.png') });
});
```

⚠ Watch the await: every action/assertion returns a Promise — a missing `await` causes false positives and flaky tests.

15 Run It & Read the Result

HANDS-ON



IN THE IDE run these in the terminal; we'll watch headed, then open the HTML report together.

```
npx playwright test --headed
```

Watch it drive a real browser

```
npx playwright test
```

Run headless (faster)

```
npx playwright test home.spec.ts
```

Run just this spec

```
npx playwright show-report
```

Open the HTML report

What to look for: green checks per project (chromium, firefox), and the report listing each test with status and failure screenshots when applicable.

Day 1 evidence: a screenshot on failure and the built-in HTML report. (Full traces and video come on Day 5.)



Screenshots

screenshot: 'only-on-failure' in config, or
page.screenshot({ path: testInfo.outputPath('x.png') }) so
runs never overwrite.



HTML report

reporter: 'html' gives a browsable report. With 'only-on-failure', failure screenshots appear automatically; manual ones are saved under test output.



IN THE IDE force a failure (break the title regex) and watch the screenshot appear in the report.

```
use: { screenshot: 'only-on-failure' } // already set in our config
```

17 Assertions: Soft vs Hard

CONCEPT

Hard — expect()

Stops the test on first failure. Use when later steps make no sense if this one failed (e.g. login must succeed).

Soft — expect.soft()

Records the failure but keeps going, so one run reports several problems. Use for independent checks on one page.

```
await expect(page).toHaveTitle(/Retail Store/);    // hard: stop if wrong
await expect.soft(banner).toBeVisible();          // soft: keep checking
await expect.soft(footer).toContainText('@ 2026'); // soft
```



From Java: expect() is JUnit/TestNG Assert (fails fast); expect.soft() is TestNG SoftAssert (collects, then reports).

18 Accessibility Basics (WCAG)

CONCEPT

Accessibility means everyone — including people using screen readers or keyboards — can use the app. We introduce the idea today; automated Axe checks arrive on Day 4.



WCAG (Web Content Accessibility Guidelines) is the standard. Its four principles spell **POUR**:

Perceivable

Content can be seen or heard (e.g. image alt text, contrast)

Operable

Usable by keyboard, not just mouse

Understandable

Clear, predictable behaviour and labels

Robust

Works with assistive tech like screen readers



IN THE IDE 5-min manual check: Tab through the page — logical order? focus visible? buttons have clear names?

GUIDED PRACTICE

Guided Exercises

Three short 'you try' tasks. Attempt each in the IDE, then we review the solution together.

~30 minutes · individual · ask for help anytime

Exercise 1 – Strengthen the Validation

YOU TRY

The test below only checks the title. A page can load the right title but still be broken. Make the validation stronger.

Your task:

- Add an assertion for the URL.
- Add an assertion that a visible heading is present.
- Keep all three using web-first expect (no sleeps).



IN THE IDE start from tests/home.spec.ts — you have the title line already.

```
await page.goto('/');  
await expect(page).toHaveTitle(/Retail Store/); // TODO: add URL + heading
```

21 Exercise 1 – Solution

SOLUTION

```
await page.goto('/');  
await expect(page).toHaveTitle(/Retail Store/);  
await expect(page).toHaveURL(/\/$/);  
await expect(page.getByRole('heading', { name: /Retail Store/i })))  
    .toBeVisible();
```

Why: title + URL + a visible element together prove the page actually rendered, not just that a tab opened with the right title.

22 Exercise 2 – Soft vs Hard

YOU TRY

You want to check the header, footer and a promo banner in one run. If one is missing, you still want to see the others. Which assertion type fits?

Your task:

- Write three checks (header, footer, banner) that all run even if one fails.
- Keep the page-load title check as a hard assertion (a precondition).



IN THE IDE use the locators given below; focus on choosing hard vs soft correctly.

```
const header = page.getByRole('banner'), footer = page.getByRole('contentinfo');  
const banner = page.getByText('Summer Sale'); // TODO: assert all three
```

```
// precondition – hard
await expect(page).toHaveTitle(/Retail Store/);

// independent checks – soft, so all run
await expect.soft(header).toBeVisible();
await expect.soft(footer).toBeVisible();
await expect.soft(banner).toBeVisible();
```

Why: the title is a precondition (hard — stop if wrong). The three independent UI checks are soft, so one run reports every missing region.

Exercise 3 — Spot the Bug

YOU TRY

This test passes even when the heading is missing. Find out why, and fix it.

```
test('home loads', async ({ page }) => {  
  await page.goto('/');  
  expect(page.getByRole('heading', { name: /Retail Store/i })).toBeVisible();  
});
```

Your task:

- Identify the bug.
- Fix it so a missing heading actually fails the test.

The bug: the `expect(...)` is missing `await`. The assertion returns a Promise that is never awaited, so the test ends before it resolves — it can never fail.

```
// X before – no await, assertion never enforced
expect(page.getByRole('heading', ...)).toBeVisible();

// ✓ after – awaited, now it really waits and verifies
await expect(page.getByRole('heading', ...)).toBeVisible();
```

⚠ **Lesson:** a missing `await` is the #1 beginner bug — it creates tests that pass for the wrong reason.

PROJECT WORK

Afternoon Lab

Build the Day-1 smoke project yourself, in five staged milestones. Each has an acceptance gate before you move on.

~4 hours · work in the IDE · trainer circulates

27 Lab Overview

PROJECT WORK

Goal: a Playwright + TypeScript project that launches the app, validates it, captures evidence, and is committed clean.

1 Setup & config

Project runs; config has one use block

2 Launch & validate

Title, URL, visible heading asserted

3 Evidence & soft checks

Screenshot + soft header/footer checks

4 Run & report

Headless green + HTML report + a11y pass

5 Commit clean

Branch, lockfile, no junk committed

Get a running project with a clean config before writing your own test.

- Clone the trainer repo and `npm ci` (or scaffold via Option B).
- Confirm `playwright.config.ts` has chromium + firefox and a single use block.
- Run the example test to confirm the toolchain works.



IN THE IDE run the bundled example spec — you should see it pass on both projects.

```
npx playwright test          # expect green on chromium + firefox
```

✓ **Gate:** example test passes headless on both browser projects.

Write your own spec: launch the app and prove it loaded correctly.



IN THE IDE create tests/home.spec.ts; assert title, URL, and a visible heading.

```
test('home launches and loads', async ({ page }) => {  
  await page.goto('/');  
  await expect(page).toHaveTitle(/Retail Store/);  
  await expect(page).toHaveURL(/\/$/);  
  await expect(page.getByRole('heading', { name: /Retail Store/i })).toBeVisible();  
});
```

✓ **Gate:** your spec passes; removing the heading assertion's target makes it fail (prove it works).

Add a screenshot and a few independent soft assertions. Note the full test signature — `testInfo` is the second argument.



IN THE IDE extend `home.spec.ts` — add `testInfo`, capture a screenshot, then soft-assert regions.

```
test('home launches and loads', async ({ page }, testInfo) => {  
  // ... goto + title/URL/heading assertions from Milestone 2  
  await page.screenshot({ path: testInfo.outputPath('home.png') });  
  
  await expect.soft(page.getByRole('banner')).toBeVisible();  
  await expect.soft(page.getByRole('contentinfo')).toBeVisible();  
});
```

✓ **Gate:** screenshot file produced under test output; both soft checks run in one pass.

31 Milestone 4 — Run & Report

LAB

Run the suite headless, open the report, and do the manual accessibility pass.



IN THE IDE run headless, open the HTML report, then Tab through the page for the a11y check.

```
npx playwright test          # headless
npx playwright show-report    # browse results + screenshot
```

Manual a11y pass: logical tab order? focus visible? buttons have accessible names?

✓ **Gate:** green headless run, report opens with your test, screenshot file exists under test output, a11y pass noted.

Land your work on a branch with good hygiene — the same way you would on a real team.



IN THE IDE branch, stage only what belongs, commit, and push for a merge request.

```
git checkout -b day1/home-smoke
git add tests/ playwright.config.ts package*.json # not test-results/
git commit -m "Day 1: home launch & validation smoke"
git push -u origin day1/home-smoke
```

✓ **Gate:** branch pushed; lockfile committed; test-results/ and node_modules/ are NOT committed.

Tip: .gitignore should already exclude node_modules/ and test-results/.



Unpinned versions

Floating versions cause 'works on my machine' — pin and lock.



npm install everywhere

Non-deterministic — prefer npm ci against the lockfile.



Missing await

Assertion never enforced — the test passes for the wrong reason.



Validating only the URL

Also assert title / a visible element — a redirect can fool you.



Committing junk

Never commit node_modules/ or test-results/.



Soft vs hard misuse

Don't soft-assert a precondition the rest of the test depends on.

Before you call a test 'green', sanity-check it. A passing test that proves nothing is worse than no test.

- ✓ **Did you watch it fail?** Break an assertion once — confirm it actually goes red.
- ✓ **Every assertion awaited?** A missing await silently disables the check.
- ✓ **More than the URL?** Title + URL + a visible element together.
- ✓ **Evidence captured?** Screenshot present; report opens and lists your test.
- ✓ **Clean commit?** Lockfile in; node_modules / results out.

Day 1 contribution areas for the Week 1 mini-capstone. The full rubric is scored out of 100 at the weekly pair demo — here is where each area comes from today:

Concept understanding	Explain architecture, context, soft vs hard	15
Hands-on implementation	Working launch & validate test in Git	30
Assertions & coverage	Title/URL + visible element, soft + hard	15
Headless / report readiness	Headless run + HTML report + screenshot	10
Git hygiene	Branch, commit, lockfile, clean tree	10
Lab completion discipline	All five milestone gates met	10
Basic accessibility observation	5-minute manual a11y pass performed	10
Total	Weekly mini-capstone score	100

- ✓ Playwright = runner + browser + isolated context + page; each test starts clean.
- ✓ Config is the single source of truth for how tests run.
- ✓ Launch with goto, validate with web-first expect (title, URL, a visible element).
- ✓ Hard assertions stop the test; soft assertions accumulate — choose deliberately.
- ✓ Always await assertions — a missing await is the #1 beginner bug.
- ✓ Accessibility (WCAG / POUR) is a first-class concern; Axe automation comes on Day 4.

Pin this for the rest of the week — the commands and the validation pattern you will reuse daily.

Commands

```
npx playwright test
npx playwright test --headed
npx playwright test --ui
npx playwright show-report
```

Validation pattern

```
// launch
await page.goto('/');
// validate
await expect(page)
  .toHaveTitle(/.../);
// evidence
await page.screenshot(
  { path: testInfo
    .outputPath('x.png') });
```

Day 1 complete

You can launch the app, validate it loaded correctly, and capture evidence — the foundation everything else builds on.



Next — Day 2: Login Workflow — locator strategies, DOM, forms and reliable interactions.